

Implementação e Benchmarking do Algoritmo de Mineração Apriori Utilizando OpenMP

N. M. de Carvalho Junior

Abstract— This article aims to show the comparison of two implementations of the algorithm of data mining Apriori for association rule of two items in a database, making use of the C++ language and the OpenMP library, being implemented one parallel and another sequential implementation, to confront the execution time of both approaches. The data for testing both implementations comes from a real database obtained on the Kaggle platform.

Keywords— OpenMP, Apriori, Parallel Computing, Benchmark

I. INTRODUÇÃO

O AVANÇO DA TECNOLOGIA, principalmente na área de computação, tem caminhado a passos largos nas últimas décadas. Em 1965, Gordon Earle Moore, declarou que a quantidade de transistores que poderiam ser colocados em uma mesma área de um processador dobraria a cada 18 meses [1]. Dessa forma, com essa elevação na quantidade dos transistores, os processadores teriam um menor consumo de energia e dissipação térmica e resultando em maior desempenho [1].

Porém, em determinado momento, mesmo com a evolução dos processadores em geral, há uma necessidade de ganho de desempenho para cálculos complexos, e uma das soluções para o aumento do poder de processamento é fazer o uso de técnicas de programação paralela [1].

A programação paralela é uma técnica na qual um problema a ser resolvido é decomposto em partes que podem ser executadas concorrentemente [2]. Nessa técnica, é empregado o uso de múltiplos processadores em ordem que cada parte do problema pode ser aplicada a cada um dos processadores disponíveis, com o objetivo de executar diferentes tarefas ao mesmo tempo [3].

Existem diversas formas de implementar o paralelismo em linguagens de programação, uma delas é a utilização de interfaces de programação de aplicações (API – *Application Programming Interface*) para a programação paralela em ambientes de multi-core, sendo mais comuns as API's de POSIX Threads e a do Open Multi-Processing (OpenMP) [4].

Para esse trabalho, foi utilizada a API OpenMP, devido à mesma possuir implementações em compiladores das linguagens C, C++ e FORTRAN [5]. O OpenMP é uma API para programação paralela explícita, constituída por diretivas de compilação, biblioteca de funções e variáveis de ambiente [5].

As diretivas de compilação são utilizadas no OpenMP para definir seções de código paralelizáveis, que torna possível a declaração de pontos e formas de paralelização [5]. O

OpenMP tem o paralelismo baseado em threads, baseado no modelo fork-join [6].

O modelo fork-join adotado pelo OpenMP faz com que os programas desenvolvidos no início da execução comecem com um *thread* mestre, na qual executa sequencialmente até a primeira definição de região paralela [6]. Ao encontrar uma região paralela, ele cria um conjunto de threads paralelos (de acordo com o número de threads definidos) e após a execução dos passos contidos na região paralela, é feita a união, ocasionando a sincronização das threads e finalização das mesmas, mantendo-se a *thread* mestre ativa [6].

A API OpenMP foi utilizada para a implementação do Algoritmo Apriori de forma paralelizada na linguagem de programação C++. O Apriori é um dos algoritmos mais famosos para Mineração de Dados, proposto por Agrawal et al em 1993, para busca de padrões (regras de associação) em bancos de dados [7].

O algoritmo Apriori é largamente utilizado na análise de cestas de compras em supermercados, para determinar como itens comprados por clientes estão relacionados [7]. O Apriori executa baseado em dois parâmetros iniciais (Suporte e Confiança) que controlam a geração das regras de associação [8].

A execução desse algoritmo pode ser custosa dependendo do tamanho do conjunto de dados, devido ao mesmo considerar três conjuntos de informações: as transações presentes em um *dataset*, a quantidade de produtos (itens) de cada transação e o número de itens candidatos (que alcançaram suporte mínimo) de uma associação [7].

A sequência de passos da execução do algoritmo é deveras simples: Dado um conjunto de transações (*dataset*), ele encontra regras para a predição de ocorrência de itens baseado na presença de outros itens na mesma transação [9].

Uma regra de associação é uma expressão na forma X para Y, onde X e Y são conjuntos de itens [9]. Para avaliar as regras encontradas, existem dois parâmetros: Suporte, no qual é a parcela de transações que contêm os produtos X e Y e Confiança, que mensura a regularidade com que Y aparece nas transações que contêm X [9].

O algoritmo, utilizando de dois parâmetros, realiza para todo item do conjunto de dados, o cálculo da confiança da regra de associação e remove os itens que são inferiores ao valor definido de Confiança e Suporte [9].

ID da Transação	Produtos Comprados
1	Pão, Leite
2	Pão, Fraldas, Cerveja, Ovos
3	Leite, Fraldas, Cerveja, Coca
4	Pão, Leite, Fraldas, Cerveja
5	Pão, Leite, Fraldas, Coca

Figura 1. Exemplo de conjunto de dados para o Apriori. [9]

Na Figura 1, tomando esse conjunto de dados como exemplo de funcionamento, temos cinco transações com cinco produtos comprados pelos clientes de um mercado. Realizando o cálculo da confiança e do suporte mínimo para cada produto do *dataset*, podem ser encontradas inicialmente três regras de associação: quem compra {leite, fralda} também compra {cerveja} (40% de suporte, 67% de confiança), quem compra {leite, cerveja} também compra {fralda} (40% de suporte, 67% de confiança) e quem compra {fralda, cerveja} também compra {leite} (40% suporte, 67% confiança) [9].

Dessa maneira, o artigo a seguir tem como objetivo a comparação do algoritmo Apriori com dados reais de um supermercado em duas implementações: sequencial (serial) e utilizando a API OpenMP para execução com múltiplos processadores/threads.

II. MATERIAIS E MÉTODOS

Ambiente de desenvolvimento

O desenvolvimento e teste das soluções, tanto a sequencial quanto a paralela, foram feitas na linguagem de programação C++, dentro do editor de código Visual Studio Code da Microsoft, no sistema operacional Windows 10 Pro de 64 bits. O compilador utilizado foi o The GNU C++ compiler, distribuído com o MinGW Minimalist GNU for Windows, na versão 8.2.0.

A limpeza dos dados obtidos para teste foi realizada utilizando a linguagem de programação Python em conjunto da biblioteca pandas, executando-a em um terminal interativo do IPython Jupyter Notebook [10].

Os testes foram realizados em computador com 8GB de memória RAM DDR4 2133MHz, um dispositivo de armazenamento Sandisk SSD 128GB e um processador Intel Core i7-7500U 2.70GHz, com função de aumento de frequência até 3.50GHz (*hyper threading*). O processador em questão possui 2 cores com suporte até 4 threads simultâneas e dispõe de memória cache L3 de 4MB, L2 de 512KB e L1 de 128KB [11].

Pré-processamento

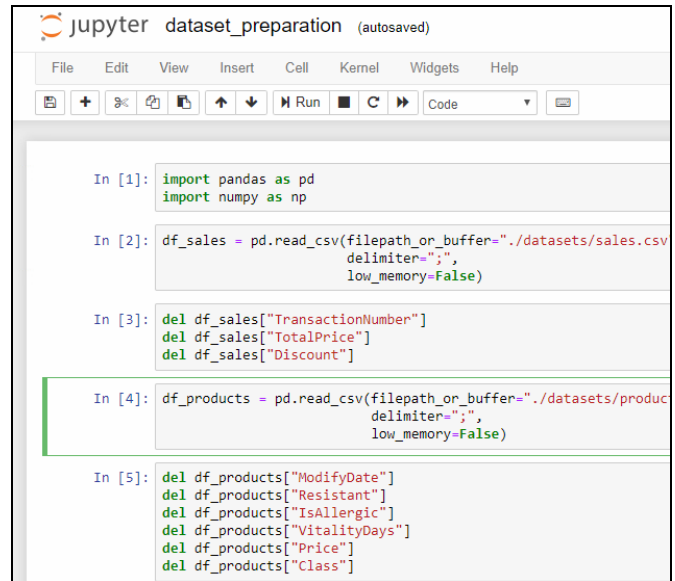
Os dados usados para o teste das implementações do Algoritmo Apriori foram obtidos do portal Kaggle®. O Kaggle é uma das mais conhecidas plataformas para competições de Data Science, fundada em 2010 por Anthony Goldbloom que foi posteriormente adquirida pela Google [12]. Este portal possui uma enorme gama de conjuntos de dados abertos, que são disponibilizados gratuitamente ao público.

O Kaggle foi escolhido pela facilidade na obtenção de conjuntos de dados e pela quantidade de conjuntos de dados de diversos tipos e temas disponíveis [13]. Dentre os *datasets* disponíveis na plataforma, foi escolhido o conjunto de dados “SalesDB Grocery Market”, no qual possui dados de uma rede de supermercados real [14].

O conjunto de dados obtido pelo portal Kaggle possuía oito arquivos em formato CSV, contendo informações sobre diversas áreas do negócio. Para nossos testes, foram utilizados os *datasets* de Produtos e de Categorias de Produtos.

Devido à natureza da origem dos dados, na qual foi oriunda de uma base de dados relacional, foram necessários ajustes para adequá-los à entrada das implementações dos algoritmos, pelo motivo desses dados possuírem atributos que não seriam utilizados.

Para o processo de limpeza dos dados obtidos, foi utilizada a linguagem Python com a biblioteca Pandas [15]. O pandas é uma biblioteca que oferece estruturas e ferramentas para análise e limpeza de dados de forma simples e concisa [16].



```

In [1]: import pandas as pd
import numpy as np

In [2]: df_sales = pd.read_csv(filepath_or_buffer='./datasets/sales.csv',
delimter=";",
low_memory=False)

In [3]: del df_sales["TransactionNumber"]
del df_sales["TotalPrice"]
del df_sales["Discount"]

In [4]: df_products = pd.read_csv(filepath_or_buffer='./datasets/product',
delimter=";",
low_memory=False)

In [5]: del df_products["ModifyDate"]
del df_products["Resistant"]
del df_products["IsAllergic"]
del df_products["VitalityDays"]
del df_products["Price"]
del df_products["Class"]

```

Figura 2. Pré-processamento utilizando Python e pandas.
Elaborado pelo autor.

Os dados foram carregados para o ambiente do pandas, onde pôde ser realizado o processo de remoção de atributos desnecessários, limpeza de dados nulos e transformação do *dataset* em outros conjuntos menores para serem empregados nos testes das implementações do Apriori, como pode ser visto na Figura 2.

O processamento dos dados resultou em três conjuntos de dados de teste, o primeiro contendo três mil linhas (3.000), o segundo com seis mil linhas (6.000) e o último com doze mil linhas (12.000).

Pré-desenvolvimento

Antes de realizar a implementação do algoritmo, foi realizado a configuração do ambiente de testes e de desenvolvimento. Para configurar o compilador da linguagem C++, bem como o gerenciamento dos pacotes de bibliotecas da API OpenMP foi utilizado o MinGW Minimalist GNU for Windows.

O MinGW é um ambiente para sistemas Microsoft Windows que fornece um conjunto de ferramentas de código aberto para desenvolvimento de aplicações em diversas linguagens de programação, dentre elas o C++, utilizando o compilador da GNU, o The GNU C++ compiler [17].

Para o desenvolvimento das soluções, bem como o emprego do OpenMP para a paralelização do código escrito em C++, foi utilizado o editor de código Visual Studio

Code. O Visual Studio Code é um editor de código fonte multiplataforma desenvolvido pela Microsoft [18].

Implementação do Apriori Sequencial

Após a configuração do ambiente de desenvolvimento, deu-se início à implementação do algoritmo Apriori. O algoritmo inicialmente foi escrito de maneira sequencial (serial).

O algoritmo feito é uma versão do Apriori desenvolvida para exibir a correlação entre dois itens dado um *dataset* que possua um conjunto de vendas realizadas em diversas transações com produtos distintos [19]. Ao executar o algoritmo, é definido dois parâmetros: O suporte mínimo para uma regra ser considerada e o nível mínimo de confiança para que uma regra seja validada.

```
int aNumTransactions = 0;
int aNumProducts = 0;
int aNumSales = 0;

string line;
ifstream file("dataset7.txt");

if (!file.is_open())
{
    cout << "Nao foi possivel executar o arquivo!\n";
    return -1;
}

getline(file, line);
aNumSales = stoi(line);

getline(file, line);
aNumProducts = stoi(line);

getline(file, line);
aNumTransactions = stoi(line);
```

Figura 3. Implementação da leitura do conjunto de dados
Elaborado pelo autor.

Como pode ser visto na Figura 3, inicialmente é feito a leitura do arquivo, que deve estar em formato de texto (TXT), como pode ser visto na figura A. Desse arquivo, é obtido o número total de produtos vendidos, o número de produtos distintos presente nesse conjunto de dados e o número de total de transações realizadas. Em seguida, é realizada a leitura de todas as vendas armazenadas no conjunto de dados.

```
int aSupportProducts[aNumProducts + 1] = {0};
int countMinSupport = 0;

for (int i = 0; i <= aNumProducts; i++)
{
    int iProductFrequency = 0;

    for (int j = 0; j < aNumSales; j++)
    {
        if (aSales[j][1] == i)
            iProductFrequency++;
    }

    aSupportProducts[i] = round(
        100 * (iProductFrequency) / aNumTransactions);

    if (aSupportProducts[i] >= aMinSupport)
        countMinSupport++;
}
```

Figura 4. Código da etapa de construção do vetor de fator de suporte
Elaborado pelo autor.

Após a leitura dos dados, inicia-se a construção de um vetor de fator de suporte, como pode ser visto na Figura 4, na qual armazenará o cálculo da frequência dos produtos em relação à quantidade de transações presentes na base de dados. Nessa etapa também é feita a contagem dos produtos que alcançaram o suporte mínimo necessário para estarem dentro das regras de associação.

Após o cálculo do fator de suporte, é feito a filtragem dos produtos que alcançaram o suporte mínimo definido previamente. Nessa etapa, é salvo em um vetor multidimensional (2D) o código identificador do produto e o nível de fator de suporte alcançado por ele. Os produtos que não atingiram o valor mínimo são descartados.

Depois de realizada a filtragem dos produtos que alcançaram o suporte mínimo, é iniciada a fase de busca por regras de associação. Nesse momento, é inicializada uma matriz que armazenará todas as regras de associação de dois itens (X, Y) encontradas pelo algoritmo, contendo quatro atributos: o código identificador do produto X, código identificador do produto Y, a frequência do produto X em relação ao produto Y e a frequência do produto Y em relação ao produto X.

```
// (productIdX, productIdY, frequencyProdX, frequencyProdY)
int **allMiningRulesFound = new int *[aNumSales];
int nextRuleFoundIndex = 0;

for (int i = 0; i < aNumSales; i++)
    allMiningRulesFound[i] = new int[4];

for (int i = 0; i < aNumSales; i++)
    for (int j = 0; j < 4; j++)
        allMiningRulesFound[i][j] = 0;

for (int i = 0; i < countMinSupport; i++)
{
    // obtemos o próximo item que tinha o suporte mínimo
    int currentMinProductID =
        aFirstMinSupportProducts[i][0];
    int currentMinProductSupport =
        aFirstMinSupportProducts[i][1];

    for (int j = 0; j < aNumSales; j++)
    {
        int nextIdTransaction = aSales[j][0];
        int nextIdProduct = aSales[j][1];

        // se o próximo produto é o mesmo do menor,
        // não faz sentido comparar
        if (nextIdProduct == currentMinProductID)
            continue;
    }
}
```

Figura 5. Código de início da fase de mineração de regras
Elaborado pelo autor.

Como pode ser visto na Figura 5, nesse momento, é feita uma varredura sobre todos os produtos que alcançaram o fator mínimo de suporte, obtendo o código identificador e o valor de suporte, e então é feita uma leitura em todas as vendas realizadas que estão no conjunto de dados. Nessa leitura, são obtidas as transações que, além de ter outros produtos que foram vendidos, também possui o produto que alcançou o fator mínimo de suporte em sua transação. Se encontrado uma transação que cumpra esse requisito, é feita uma contagem da frequência de quando no conjunto de dados aparece esses dois produtos (o produto que alcançou o fator de suporte e os outros produtos vendidos).

```

int nextFrequencyTogether = 0;
for (int k = 0; k < aNumSales; k++)
{
    if (aSales[k][1] == nextIdProduct)
    {
        for (int l = 0; l < aNumSales; l++)
        {
            if (aSales[l][0] == aSales[k][0]
                && aSales[l][1] == currentMinProductID)
            {
                nextFrequencyTogether++;
                break;
            }
        }
    }
}

int nextSupportTogether = round(100 * (nextFrequencyTogether)
                                / aNumTransactions);

// descartamos as regras que tenham a menor confiança)
double xyConfidence = ((double)nextSupportTogether /
                       (double)currentMinProductSupport);

if (xyConfidence < ((double)aMinConfidence / 100))
    continue;

```

Figura 6. Código da seção de cálculo da confiança das regras
Elaborado pelo autor.

Então, é realizado o cálculo de confiança, que é razão da contagem da frequência que aparece os dois produtos pelo número total de transações presentes no conjunto de dados. Conforme é exibido na Figura 6, caso o resultado do cálculo de confiança não atinja o mínimo estipulado na execução do algoritmo, a regra é descartada. Após essa etapa, é adicionada a regra à matriz de regras encontradas.

Ao fim do processamento é exibido a lista de regras encontradas para os produtos X e Y, bem como a frequência em que ambos aparecem no conjunto de dados.

Paralelização do Algoritmo Apriori

Para desenvolver a versão paralelizada do algoritmo Apriori, foi utilizado como base o algoritmo desenvolvido de forma sequencial, devido ao OpenMP possuir diretivas de compilação, que permite a facilidade de conversão de um código serializado para um código paralelo.

Nessa versão do Apriori desenvolvida, não foram necessárias alterações na estrutura do código, sequer a reescrita das rotinas previamente desenvolvidas, apenas a inclusão das diretivas `#pragma omp parallel` em algumas partes do código para definir uma seção paralelizável e a `#pragma omp for`, para ser possível a divisão da carga de trabalho do laço `for` em mais de uma thread [20].

```

#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i <= aNumProducts; i++)
    {
        int iProductFrequency = 0;

        for (int j = 0; j < aNumSales; j++)
        {
            if (aSales[j][1] == i)
                iProductFrequency++;
        }

        // calculamos o suporte desse produto
        aSupportProducts[i] = round(100 * (iProductFrequency)
                                    / aNumTransactions);

        // contamos quantos produtos tem o suporte mínimo
        if (aSupportProducts[i] >= aMinSupport)
            countMinSupport++;
    }
}

```

Figura 7. Código da seção de cálculo de confiança com diretivas OpenMP
Elaborado pelo autor.

A diretiva de paralelização (`omp parallel`) foi utilizada acompanhado da diretiva de divisão de carga de laço “for” (`omp for`) nas etapas de criação da matriz de fator de suporte dos produtos, inicialização da matriz de produtos que alcançaram o suporte mínimo e no laço mais externo da fase de mineração de regras de associação.

De forma a modificar a quantidade de threads utilizadas pela API OpenMP, foi utilizado a variável de ambiente `OMP_NUM_THREADS` para definir a quantidade de threads a serem usadas pelo código paralelizado em tempo de execução [20].

III. RESULTADOS E DISCUSSÃO

Para validar as implementações do algoritmo, foram realizados testes utilizando os conjuntos de dados obtidos do Kaggle e pré-processados previamente. Em ambas as implementações (paralela e serial) foram definidos os parâmetros de fator mínimo de suporte em 10% e a porcentagem mínima de confiança para 10%, visando que o algoritmo executasse a maior quantidade possível de passos.

Após a definição dos parâmetros, foram executadas as implementações para os três conjuntos de dados e foi feita a primeira comparação, entre o código paralelo executando em quatro threads e o código serial, em apenas um thread. Para a obtenção dos tempos de execução, foram utilizadas as funções `clock` para a implementação serial e `omp_get_wtime` para a implementação com o OpenMP.

O resultado da comparação entre ambos pode ser visto na Tabela I, na qual contém as execuções, o tamanho dos conjuntos de dados e o tempo em segundos de cada execução.

TABELA I
COMPARAÇÃO ENTRE EXECUÇÕES DE 1 VS. 4 THREADS

#	n = 3000		n = 6000		n = 12000	
	Serial	OMP	Serial	OMP	Serial	OMP
1	0.334	0.134	3.091	1.627	41.366	18.510
2	0.336	0.137	3.074	1.518	38.824	18.524
3	0.284	0.133	3.057	1.480	38.907	18.638
4	0.292	0.128	3.189	1.577	39.656	18.844
5	0.281	0.142	3.686	1.534	39.269	18.804
6	0.279	0.134	3.149	1.492	39.203	18.446
7	0.372	0.131	3.214	1.494	41.914	18.925

A redução de tempo de execução em segundos, se comparadas ambas as implementações, varia entre 47% a 44% a favor do OpenMP, quando comparada a execução com o dataset de tamanho 12,000. Se comparado com os outros dois conjuntos de dados, o ganho varia entre 35% a 49% para o dataset de tamanho 3,000 e de 41% a 52% para o dataset de tamanho 6,000.

TABELA II
MÉDIA DE TEMPO DE EXECUÇÃO DOS DATASETS

	n = 3000	n = 6000	n = 12000
Sequencial	0.311	3.074	39.699
OMP 2 Threads	0.170	2.104	25.823
OMP 3 Threads	0.151	1.737	20.764
OMP 4 Threads	0.134	1.541	18.670

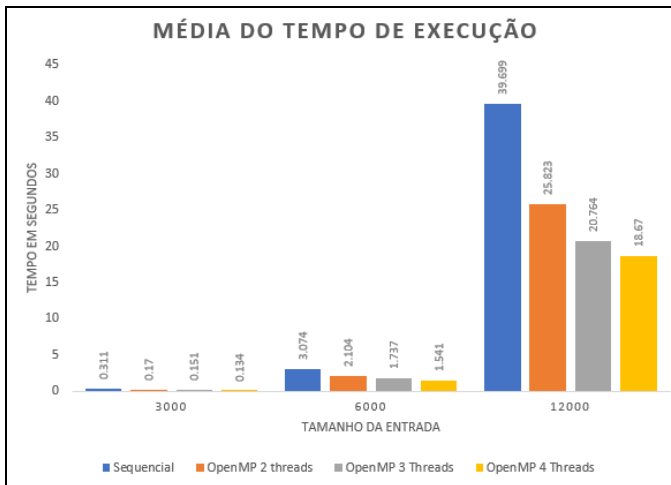


Figura 8. Gráfico da Média de Tempo de Execução
Elaborado pelo autor

Na tabela II, quando comparado a média do tempo de execução dos três datasets (três mil linhas, seis mil linhas e doze mil linhas) é nítida a queda no tempo de execução, principalmente quando executado em múltiplas *threads*. A medida em que cresce a quantidade de elementos presentes em um conjunto de dados, aumenta a quantidade de comparações que deverão ser realizadas, e nesse cenário que é perceptível o aumento de desempenho com a implementação em OpenMP, se comparado com a versão serial, conforme pode ser visualizado na Figura 8.

TABELA III
COMPARAÇÃO EXECUÇÃO DE MÚLTIPLAS THREADS (OpenMP)

#	n = 6000				n = 12000			
	Número de Threads				Número de Threads			
	1	2	3	4	1	2	3	4
1	3.08	1.96	1.63	1.62	37.97	24.28	19.65	18.51
2	3.01	1.95	1.56	1.51	37.86	24.44	19.62	18.52
3	3.02	1.96	1.51	1.48	37.95	24.27	19.59	18.63
4	3.01	1.94	1.54	1.57	38.02	24.82	19.65	18.84
5	2.95	1.97	1.59	1.53	37.98	24.40	19.63	18.80
6	3.12	1.93	1.63	1.49	38.05	24.36	19.74	18.44
7	3.06	1.96	1.55	1.49	37.94	24.41	19.48	18.92

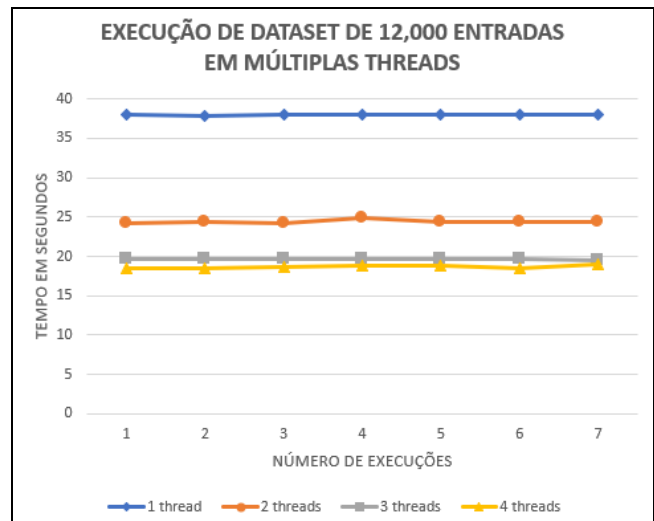


Figura 9. Gráfico dos tempos de execuções em múltiplas threads
Elaborado pelo autor

Quando comparados em múltiplas threads os conjuntos de dados de maior tamanho, com 6,000 linhas e 12,000 linhas respectivamente, como pode ser visto na Tabela III, a diminuição de tempo é gradual à medida que é incrementado o número de threads disponíveis para o OpenMP utilizando a variável de ambiente `set OMP_NUM_THREADS`.

Conforme representado na Figura 9, as diferenças significativas de aumento de desempenho são perceptíveis em 1, 2 e 4 threads. É possível também verificar na Figura 9 que o tempo de execução é similar quando o algoritmo paralelizado é executado com 3 ou 4 threads, tendo uma diferença de em média 1 segundo entre ambas execuções.

IV. CONCLUSÃO

A implementação do algoritmo Apriori, e a comparação da execução do mesmo com as versões sequenciais e paralelizadas utilizando o OpenMP foram bastante promissoras. O ganho de tempo em segundos de execução entre ambas as abordagens variou em média entre 35% a 52% a menos de tempo comparando a versão serial do código e a implementação paralelizada, dependendo da quantidade de threads reservadas para a execução e do tamanho do conjunto de dados.

Dentre todas as etapas do projeto, a que demandou maior esforço foi a etapa de obtenção e limpeza do conjunto de dados obtido e a adequação do mesmo para as versões do algoritmo codificado.

Como trabalhos futuros, pode-se destacar a implementação e comparação (benchmarking) do algoritmo Apriori utilizando outras técnicas e padrões de programação paralela com GPU/CPU como OpenCL e Nvidia CUDA.

REFERÊNCIAS

- [1] CIPOLI, Pedro. CanalTech, 2015. O que é a Lei de Moore?. Available in: <<https://canaltech.com.br/mercado/O-que-e-a-Lei-de-Moore/>>
- [2] MARTINS, Simone de Lima. Slides baseados no Tutorial Introduction to Parallel Computing (https://computing.llnl.gov/tutorials/parallel_comp/). Available in: <<http://www2.ic.uff.br/~simone/labprogparl/contaulas/conceitos.pdf>>
- [3] SATARDEY, Srushtee. Geeks for Geeks, Introduction to Parallel Computing. Available in: <<https://www.geeksforgeeks.org/introduction-to-parallel-computing/>>.
- [4] ANDRADE, Gabriella Lopes. CERA, Márcia Cristina. Comparação do uso de OpenMP e Pthreads em uma Paralelização de Multiplicação de Matrizes. Available in: <<http://www.lbd.dcc.ufmg.br/colecoes/eradrs/2016/056.pdf>>
- [5] SILVA, Michel de A. Aplicação do Algoritmo Apriori para uma base de dados de ictioplâncton em um reservatório de água doce da Amazônia Legal. Available in: <<http://www.lbd.dcc.ufmg.br/colecoes/eniac/2013/0038>>.
- [6] CENIAC, 2014. Apostila de Treinamento, Introdução ao OpenMP. Available in: <https://www.cenapad.unicamp.br/servicos/treinamentos/apostilas/apostila_openmp.pdf>.
- [7] BARANAUSKAS, José Augusto, 2007. Departamento de Física e Matemática FFCLRP USP. Regras de Associação. Available in: <http://dcm.ffclrp.usp.br/~augusto/teaching/ami/AM_I_RegrasAssociacao.pdf>
- [8] PICHILIANI, Mauro. iMasters, 2008. Data Mining na Prática: Regras de Associação. Available in: <<https://imasters.com.br/back-end/data-mining-na-pratica-regras-de-associacao>>
- [9] BOGORNY, Vania, 2000. Apresentação adaptada do material do livro: Introduction to Data Mining Tan, Steinbach e Kumar. Available in: <www.inf.ufsc.br/~vania/teaching/INE5644/RegrasDeAssociacao.pptx>
- [10] Dados e Decisões, 2018. O Jupyter Notebook, o que é?. Available in: <<https://dadosedecisoes.com.br/o-jupyter-notebook-o-que-e/>>.
- [11] Notebook Check, 2019. Intel Notebook Processor Core i7 Specs. Available in: <<https://www.notebookcheck.info/Intel-Core-i7-7500U-Notebook-Processor.211248.0.html>>
- [12] RIBEIRO, Thiago. Tirando Lições de Tudo, 2018. Kaggle – O que é? Como funciona?. Available in: <<http://tirandolicoesdetudo.com.br/kaggle-o-que-e-como-funciona/>>
- [13] VASCONCELLOS, Paulo, 2017. Os 7 melhores sites para encontrar datasets para projetos de Data Science. Available in: <<https://paulovasconcellos.com.br/os-7-melhores-sites-para-encontrar-datasets-para-projetos-de-data-science-8a53c3b48329>>
- [14] KAGGLE, 2017. SalesDB Grocery Market Dataset Description Context. Available in: <<https://www.kaggle.com/marcinex1423/salesdb-grocery>>
- [15] About the pandas project. Available in: <<https://pandas.pydata.org/about.html>>
- [16] SILVA, Daniel Fernandes. LABS BAWI, 2017. Introdução ao Pandas. Available in: <<https://labs.bawi.io/introdu%C3%A7%C3%A3o-ao-pandas-ea6c532470d5>>.
- [17] DIGITANDO CÓDIGOS, 2011. MinGW – Minimalist GNU for Windows. Available in: <<https://digitandocodigos.wordpress.com/2011/02/09/mingw-minimalist-gnu-for-windows/>>
- [18] DIONISIO, Edson José. DEVMEDIA, 2016. Introdução ao Visual Studio Code. Available in: <<https://www.devmedia.com.br/introducao-ao-visual-studio-code/34418>>
- [19] MELO, Humberto Gustavo de. DEVMEDIA, 2008. Algoritmo Apriori no MySQL utilizando procedure. Available in: <<https://www.devmedia.com.br/algoritmo-apriori-no-mysql-utilizando-procedure/9930>>
- [20] SEN, Arpan. IBM, 2012. Aprendendo a usar a estrutura OpenMP com GCC. Available in: <<https://www.ibm.com/developerworks/br/aix/library/au-aix-openmp-framework/index.html>>